Report: Optimising NYC Taxi Operations

1. Data Preparation

1.1. Loading the dataset

1.1.1. Sample the data and combine the files

- First and foremost imported all required libraries: numpy, pandas, matplotlib and seaborn
- Mount Google drive and ensure all parquet files are placed in Google drive. In my case I have placed all parquet files in EDA Assignment folder
- For taking the data sample, I have taken 0.07 percentage of entries in every hour for each date.
- Have written a method **sampleData**(df) which does the above operation by segregating the date and time for **tpep_pickup_datetime** and grouping the data based on date and hour and sampling 0.07% of data:
- The same method (**sampleData**) is called for every parquet file and merged with the main file once the data is sampled: Data merging is done using the combine() method of pandas and held in a data frame:
- Total of 2654955 records and 22 columns are sampled with 7 percentage and the final data is saved in the same google driver folder:

2. Data Cleaning

Upon initially reviewing the data using **df.info()**, below observations were made:

- It is a 2-dimensional array
- Passenger_count and RatecodeId column is in float64
- There are two airport_fee columns in the dataset
- Around 298735 Airport_Fee column is empty, which accounts to around 12% of the data

2.1. Fixing Columns

2.1.1. Fix the index

Dropped column **mta_tax** as it is 0.5 (standard) for majority of the records (2629739).

2.1.2. **improvement_surcharge** is dropped as it is 1 for majority of the records(2651798)

2.1.3. Dropped column **store_and_fwd_flag** as it is not of much significance

2.1.4. Changed the data type of **passenger_count** to integer and rounded off **tolls_amount** to 2 decimal places to avoid exponential values

2.1.5. Reset index using **reset_index** method after doing all the fixing of columns

2.1.6. **Combine the two airport_fee columns**
Since airport_fee has lesser non nulls when compared to Airport_fee, merge Airport_fee with airport_fee where ever it is null: and drop column airport_fee

2.1.7. For the below columns that has negative value, replace them with the mean:

- extra (4 records with negative)
- total_amount(123 records with negative value
- Airport_fee (24 records with negative value)
- Congestion_surcharge (91 records with negative value)

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

- Missing proportion is 0.034187 for the below columns: This was achieved using isna().mean() method:
  - Passenger_count
  - RatecodeID
  - Store_and_fwd_flag
  - Congestion_surcharge
  - Airport_fee

2.2.2. Handling missing values in passenger_count

- Impute passenger_count with **Mode**, so that there is no record with passenger_count as NAN. Also convert the data type of passenger_count from float to int:

2.2.3. Handle missing values in RatecodeID

- There are no records with fare_amount negative, when we compare the RatecodeID for total_amount negative we see that majority of the RatecodeID uses standard rate
- Impute **RatecodeID** with **Mode**, so that there is no record with **RatecodeID** as NAN. Also convert the data type of **RatecodeID** from float to int:

2.2.4. Impute NaN in congestion_surcharge

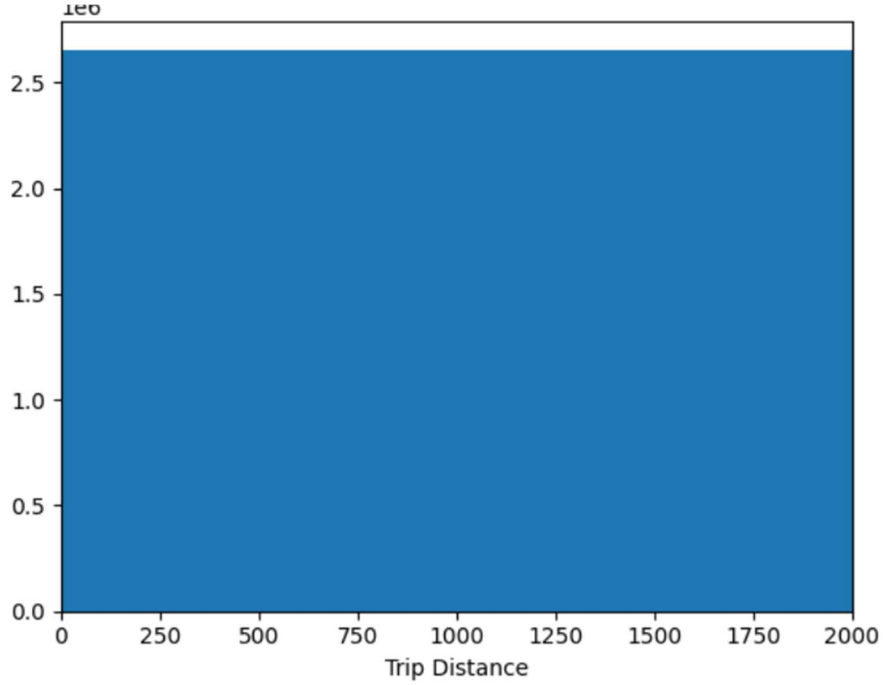
- Impute **congestion_surcharge** with **Mode**, so that there is no record with **congestion_surcharge** as NAN.
- Same was done for "**Airport_fee**" column, and ensured there are no NAN values in the entire dataframe:

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

Outliers in below cases were identified:

- passenger_count – passenger_count > 6 (31 records)
- fare_amount > 14000 (1 record)
- trip_distance is zero and fare_amount > 300 (50 record)
- trip_distance is zero and fare_amount is zero and pickup location is different from drop location (98 records)
- trip_distance more than 250 miles (63 records)
- payment_type is zero (90865)
- trip_distance is going upto 2000, which is truly an outlier



Made the below changes to handle Outliers

- Removed records with passenger_count is more than 6
- Drop records where trip_distance = 0 and fare_amount >300
- Replace fare_amount with median for entries where fare_amount > 300
- Drop records where trip_distance = 0 and fare_amount = 0 and Pickup location is not same as drop location
- Replace trip_distance with median for entries where trip_distance >250

3. Exploratory Data Analysis

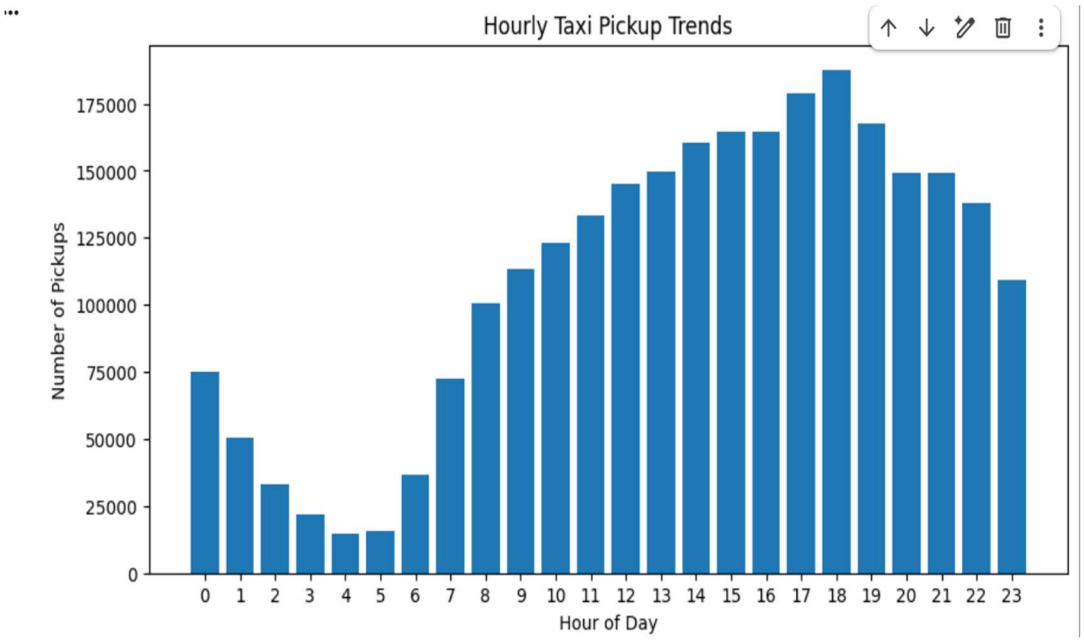
3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

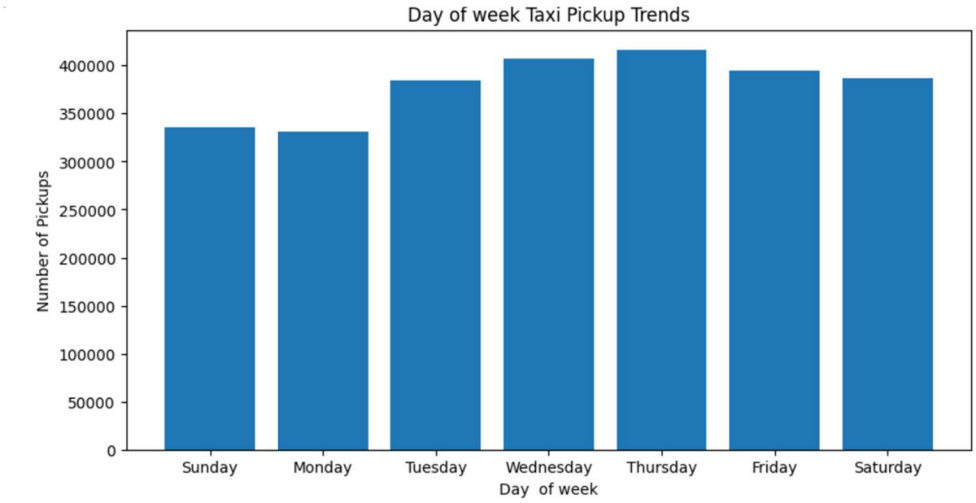
- Numeric Columns - 'VendorID', 'passenger_count', 'trip_distance', 'RatecodeID', 'PULocationID', 'DOLocationID', 'payment_type', 'fare_amount', 'extra', 'tip_amount', 'tolls_amount', 'total_amount', 'congestion_surcharge', 'hour', 'Airport_fee'
- Categorical Columns - None
- Datetime Columns - 'tpep_pickup_datetime', 'tpep_dropoff_datetime', 'date'

3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

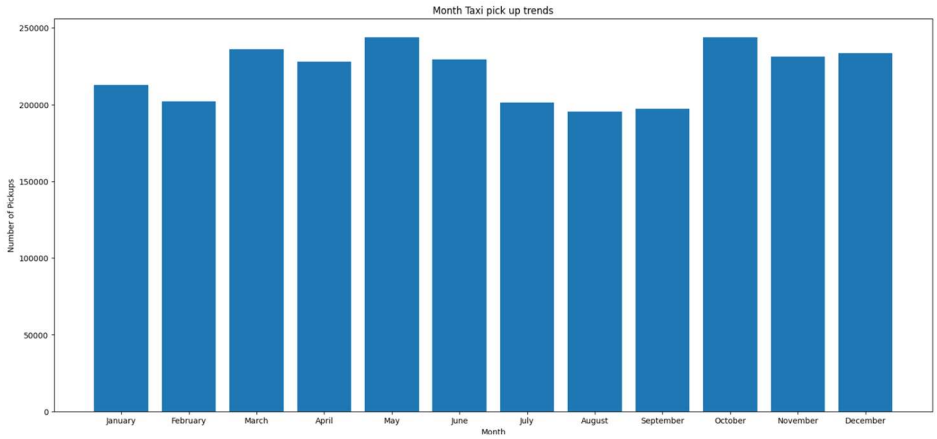
- Taxi pickup by Hours: Pickups are more between 5 to7 pm:



- Taxi pickup by Days of the week: Pickups are more on Wednesday and Thursday when compared to other days



- Taxi pickup by Month:



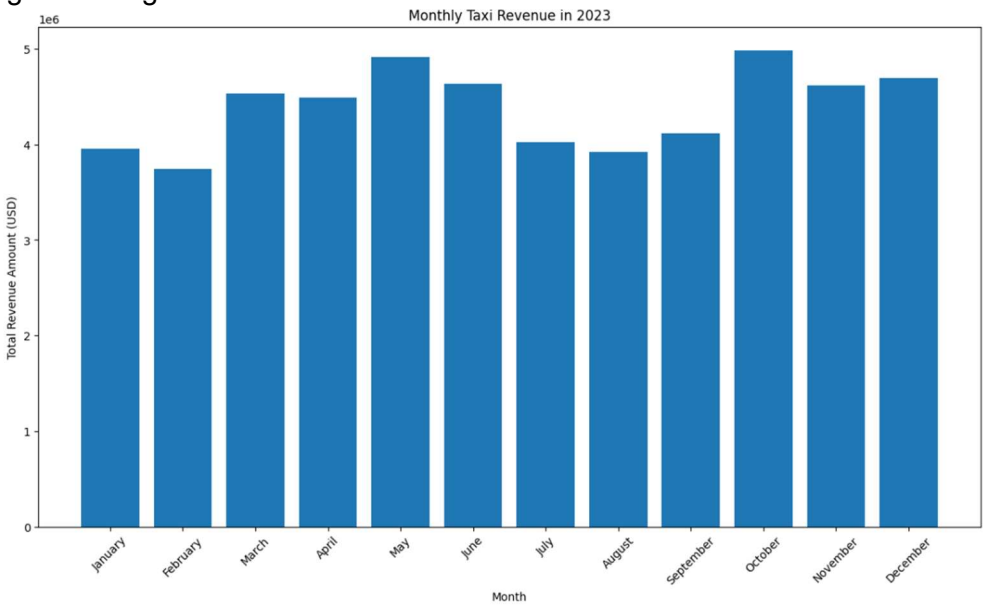
3.1.3. Filter out the zero/negative values in fares, distance and tips

- Fare_amount – 815 zeroes
- Tip_amount – 609669 zeroes
- Total_amount – 358 zeroes
- Trip_distance – 52629 zeroes

Filtered data to hold entries where fare_amount is more than zero and total_amount is more than zero. Left the trip_distance and tip_amount as it is, as it may be legitimate:

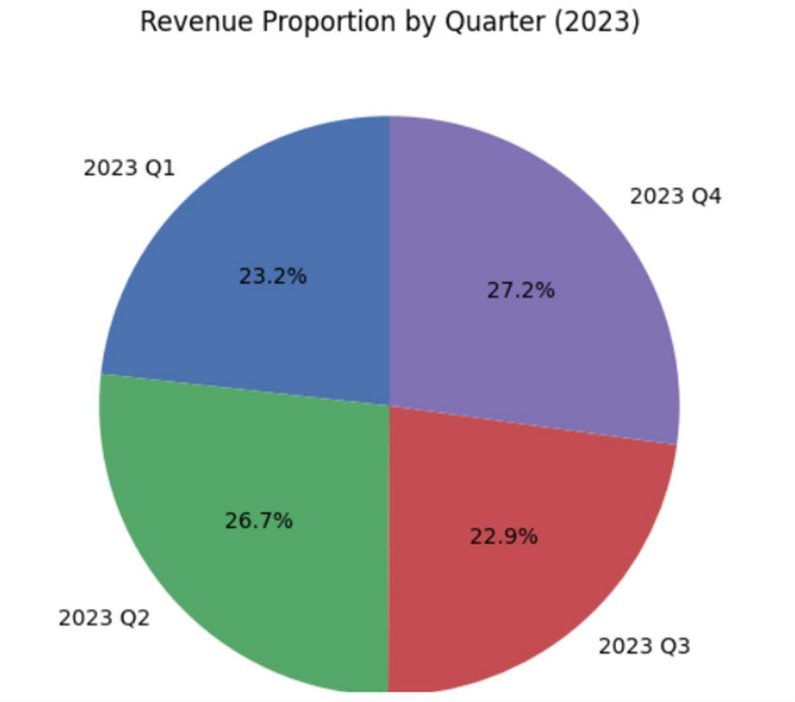
3.1.4. Analyse the monthly revenue trends

Monthly revenue is on an average more than four million dollars and with highest being in the month of October:



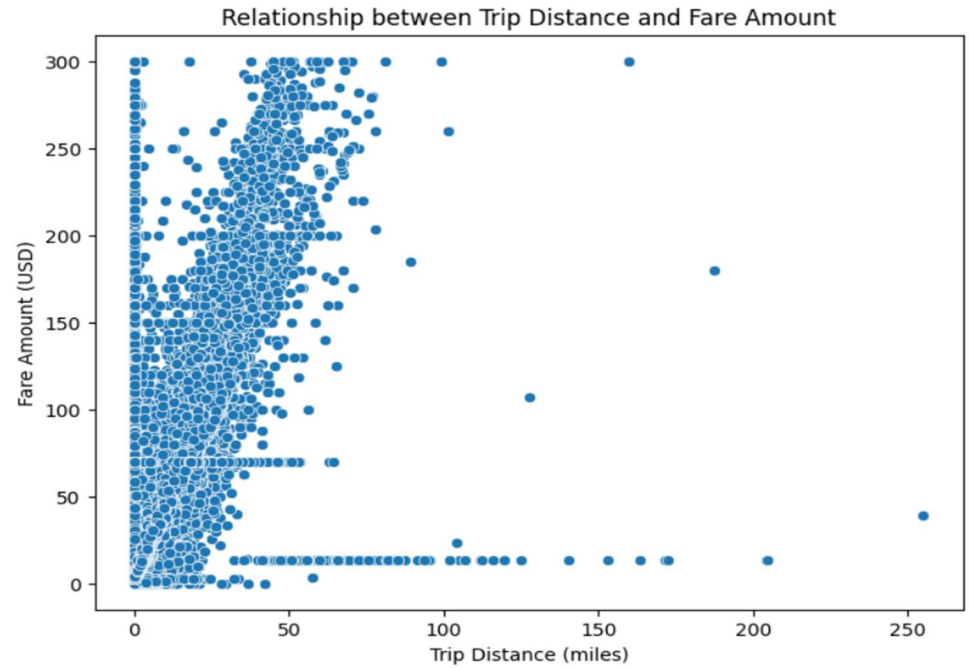
3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

Revenue proportion being relatively high in Q2 and Q4 when compared to Q1 and Q3.



3.1.6. Analyse and visualise the relationship between distance and fare amount

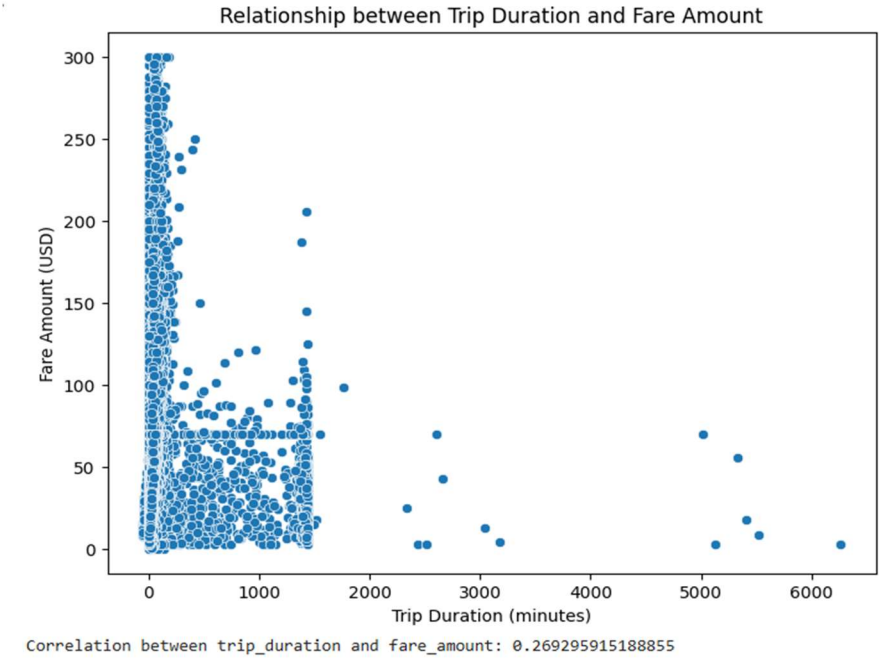
Since is correlation value is close to 1, it is very clear that distance and fare amount has positive linear relationship. When the trip distance is more the fare amount is more



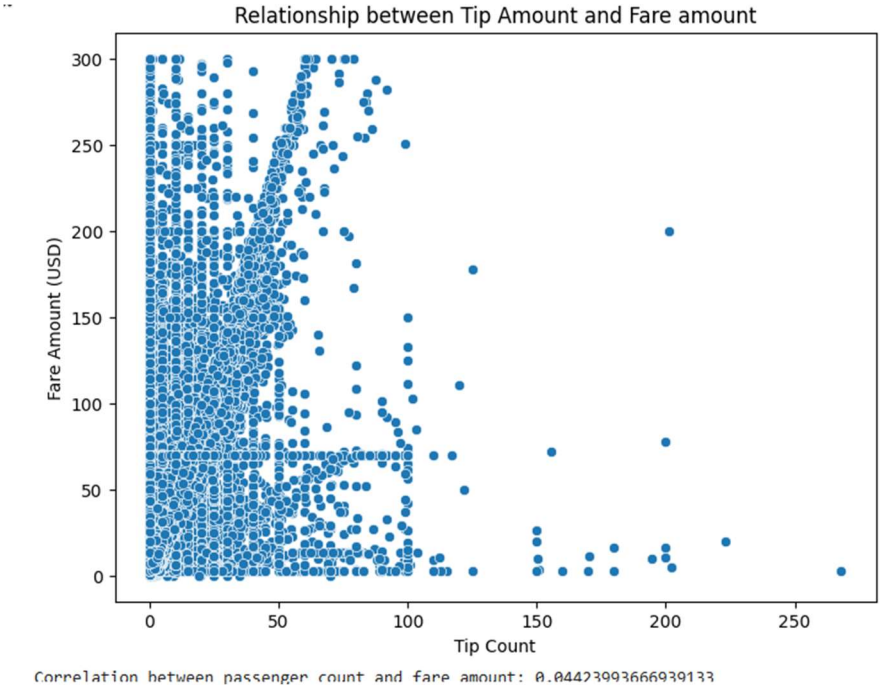

```
Correlation:  0.9027576396604426
```

3.1.7. Analyse the relationship between fare/tips and trips/passengers

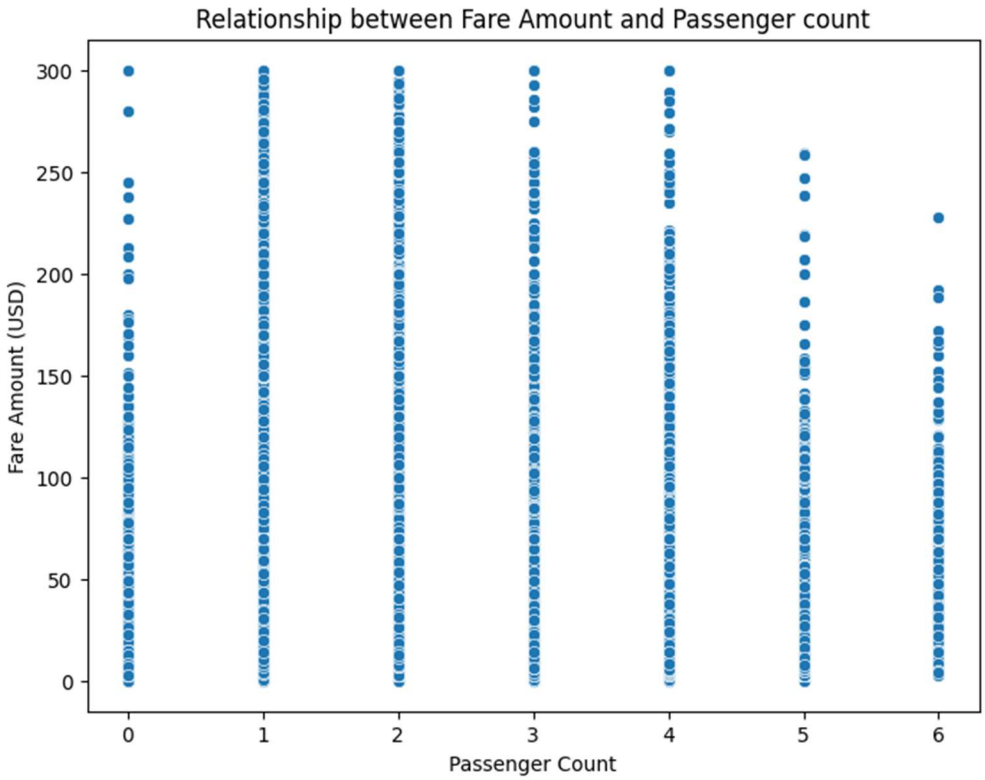
There is no positive corelation between trip duration and fare amount, may be considering cases like traffic, free rides, discounts etc




```
Correlation between trip_duration and fare_amount: 0.269295915188855
```



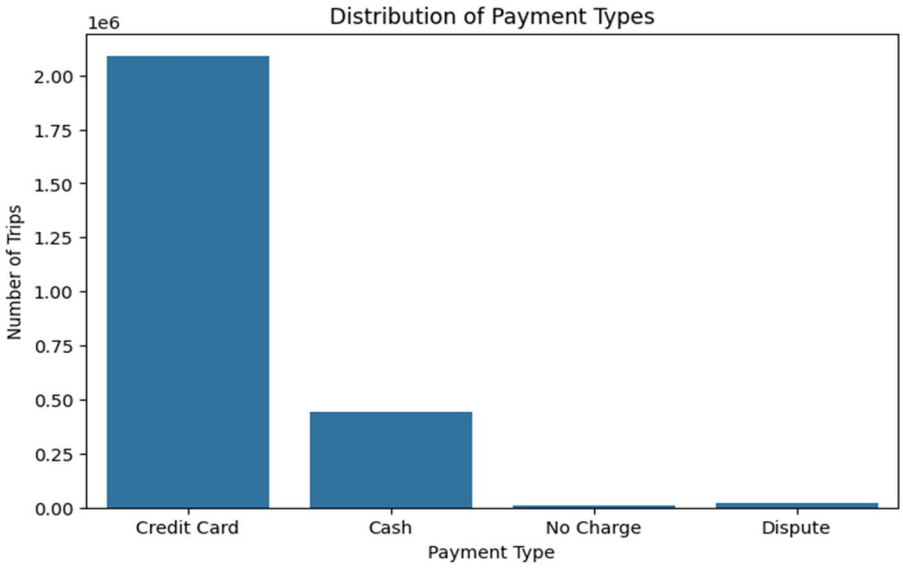
Correlation between passenger_count and fare_amount is very less, when the passenger_count is 5 and above, the fare_amount reduces




```
Correlation between passenger_count and fare_amount: 0.04423993666939133
```

3.1.8. Analyse the distribution of different payment types

- Majority of the passengers use Credit card as the payment type



3.1.9. Load the taxi zones shapefile and display it

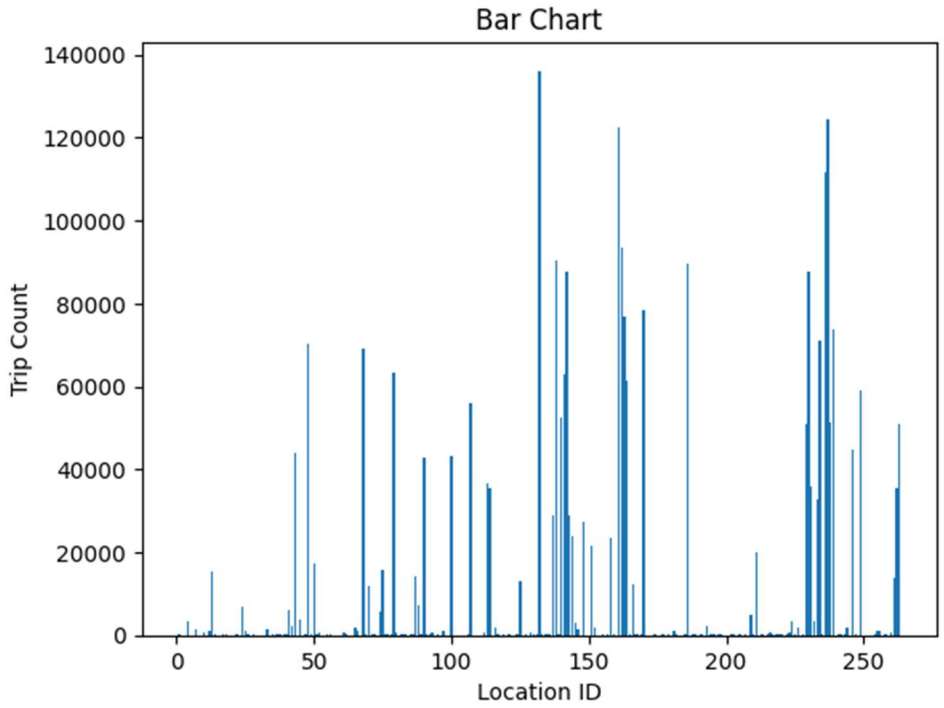
- Used geopandas library to read shapefile and load the data into dataframe:
- ObjectID, Shape_Leng, Shape_Area, zone, LocationID, borough and geometry fields were loaded

3.1.10. Merge the zone data with trips data

- Used inner join to merge trips dataframe with zones dataframe on PULocationId (from trips dataframe) with that LocationId (from zones dataframe)

3.1.11. Find the number of trips for each zone/location ID

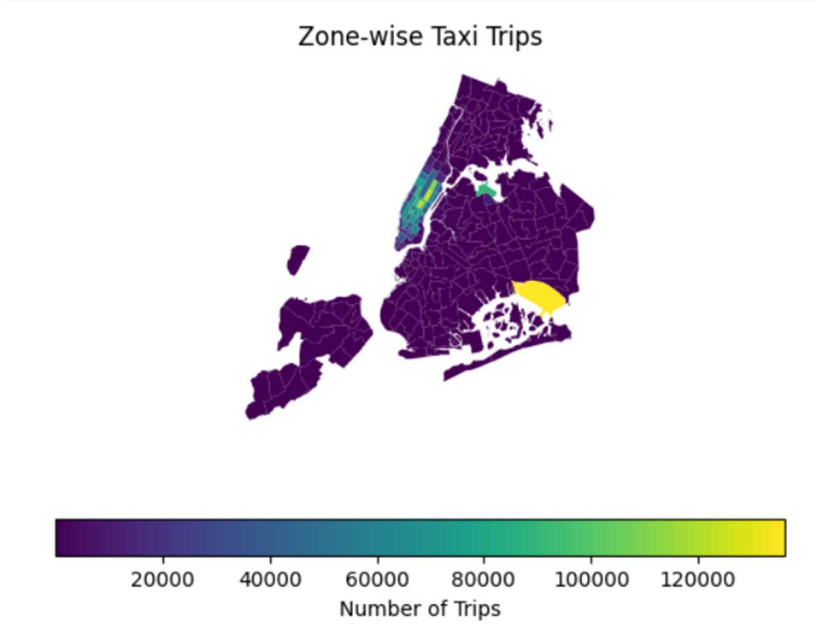
- Grouped data by locationID to find the number of trips with respect to each location:



3.1.12. Add the number of trips for each zone to the zones dataframe

- Merged zones dataframe with trips dataframe on LocationID to fetch the number of trips for each zone

3.1.13. Plot a map of the zones showing number of trips



3.1.14. Conclude with results

- Taxi pickups are generally high between 5 pm to 7 pm, pickups are more on Wednesdays and Thursday. Month on month there has been good pickup with good revenue flow thus resulting in revenue generation of more than 21% quarter on quarter:

3.2. Detailed EDA: Insights and Strategies

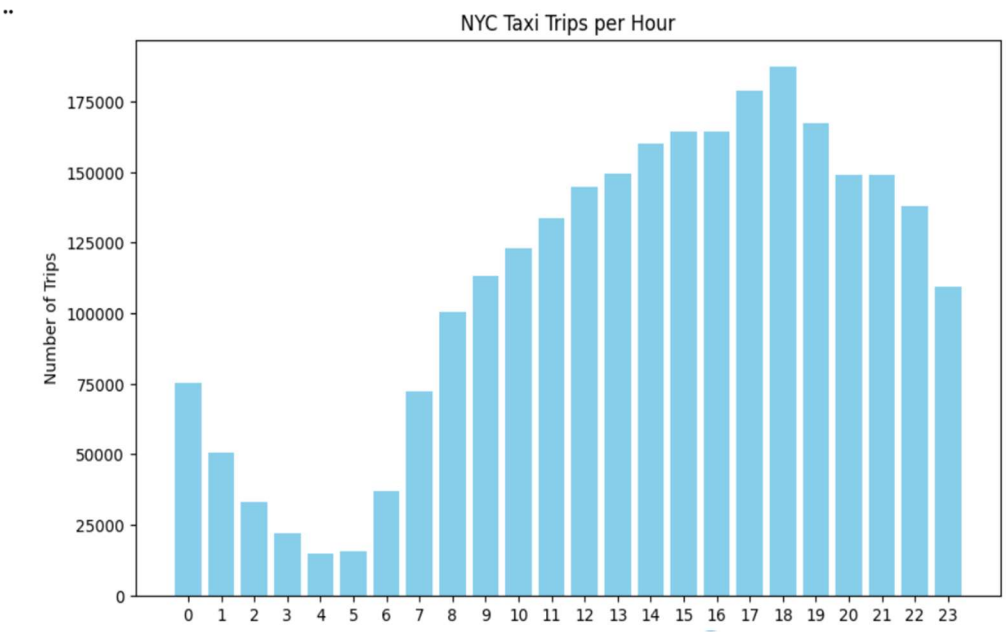
3.2.1. Identify slow routes by comparing average speeds on different routes

- Find routes which have the slowest speeds at different times of the day
- Determine trip duration in hours by finding the difference between the dropoff time and pickup time
- Calculate Speed = trip_distance/trip_duration
- Calculate average speed grouping by route and hour
- Sort the dataframe by hour and speed to get the slowest routes

Some of the slowest routes have taken around 250 miles per hour:

3.2.2. Calculate the hourly number of trips and identify the busy hours

- This data clearly shows that the peak volume is between 5 pm to 7 pm and the volume gradually increases starting from 7 am in the morning:

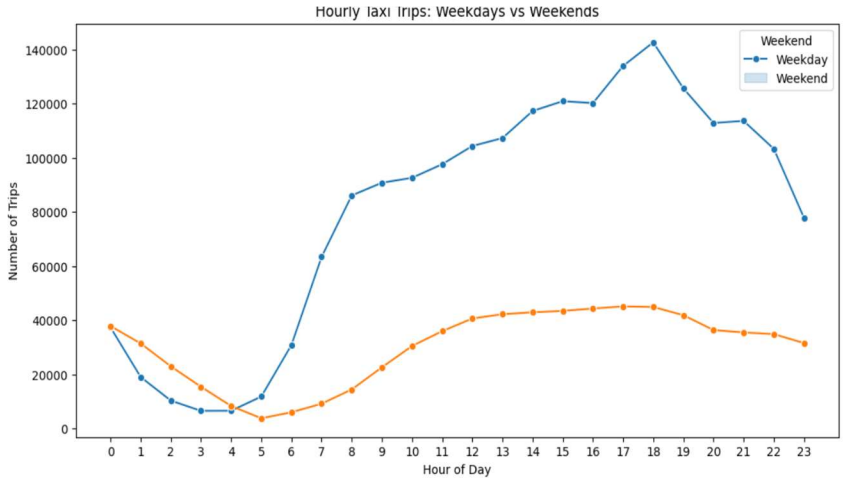


3.2.3. Scale up the number of trips from above to find the actual number of trips

- From the above daraframe sort by trip_count in descending order to find the actual number of trips in five busiest hours. We see that there were around 187460 trips at 6 pm and 178958 trips at 5 pm and 167457 trips at 7 pm.

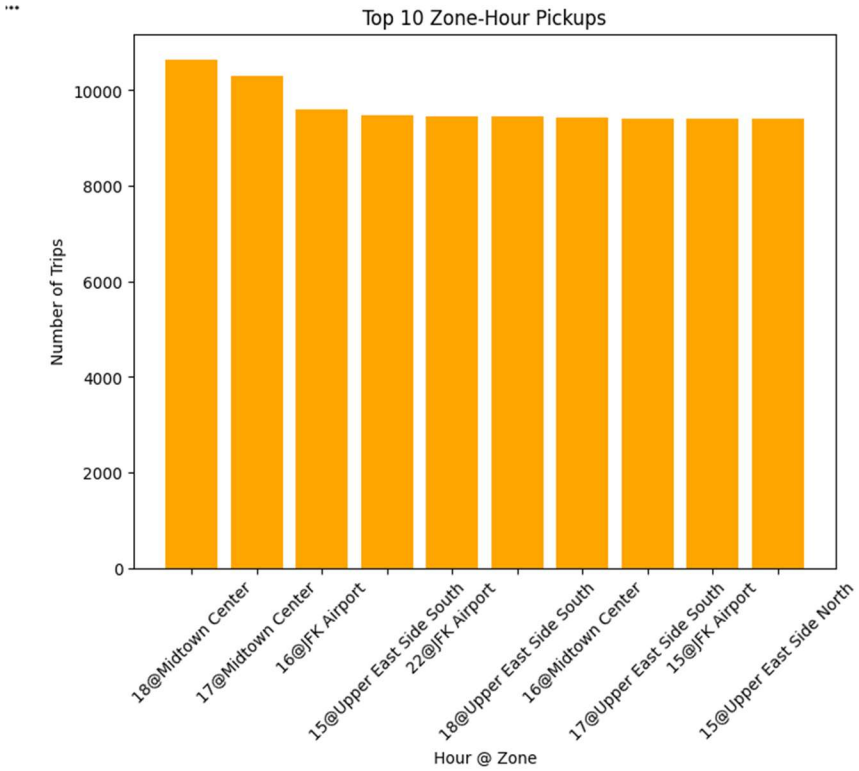
3.2.4. Compare hourly traffic on weekdays and weekends

- It is very evident that the number of trips during weekdays is way too high when compared to that of weekend:

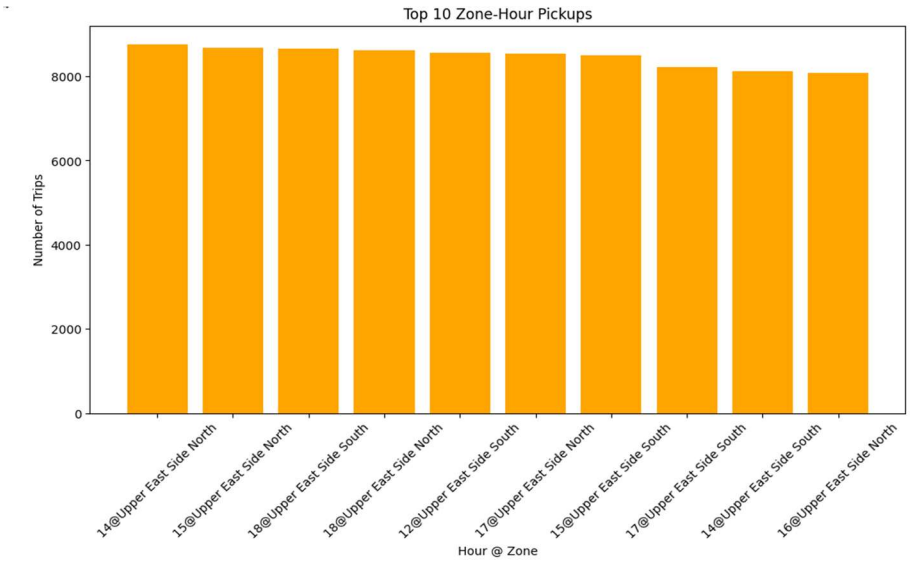


3.2.5. Identify the top 10 zones with high hourly pickups and drops

Pickups are high from 18@Midtown Center, 17@Midtown Center and from Airports:



Drops are high to 14,15 and 18@Upper East Side North and South



3.2.6. Find the ratio of pickups and dropoffs in each zone

- Group by PULocationID and DOLocationID and get the pickup_count and dropoff_count
- Calculate the pickup_dropoff ratio by dividing the pickup_count by dropoff_count
- Find the top 10 and bottom 10 pickup_dropoff_ratio by sorting the data in Ascending and Descending order:
- Merge the dataset with zones to get the actual location name:
- Final data looks as below:

Top 10 Zones (High Pickup/Dropoff Ratio):

	zone	LocationID	pickup_count	dropoff_count	pickup_dropoff_ratio
0	East Elmhurst	70.0	11770.0	1473.0	7.990496
1	JFK Airport	132.0	136153.0	31921.0	4.265311
2	Rikers Island	199.0	4.0	0.0	4.000000
3	LaGuardia Airport	138.0	90398.0	34268.0	2.637971
4	Penn Station/Madison Sq West	186.0	89760.0	57942.0	1.549135
5	Greenwich Village South	114.0	35321.0	25420.0	1.389496
6	Central Park	43.0	43802.0	32134.0	1.363104
7	West Village	249.0	58938.0	44039.0	1.338314
8	Midtown East	162.0	93465.0	75174.0	1.243316
9	Midtown Center	161.0	122471.0	103163.0	1.187160

Bottom 10 Zones (Low Pickup/Dropoff Ratio):

	zone	LocationID	pickup_count	dropoff_count	pickup_dropoff_ratio
0	West Brighton	245.0	1.0	41.0	0.024390
1	Newark Airport	1.0	325.0	8088.0	0.040183
2	Broad Channel	30.0	1.0	24.0	0.041667
3	Breezy Point/Fort Tilden/Riis Beach	27.0	2.0	45.0	0.044444
4	Mariners Harbor	156.0	2.0	41.0	0.048780
5	Windsor Terrace	257.0	57.0	1120.0	0.050893
6	Stapleton	221.0	3.0	53.0	0.056604

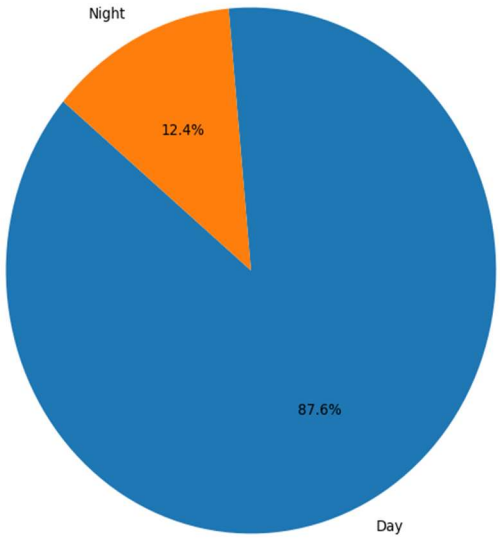
3.2.7. Identify the top zones with high traffic during night hours

- First and foremost defined the night dataframe with night hours between 11 pm to 4 am and filtered all trips that fell within this range:
- Calculated the pickup_count and dropoff_count grouping by PULocationID and DOLocationID
- Merged the dataset with zones to get the location name:
- Added the total pickup_count and dropoff_count to get the total_night_trips:
- Sorted the data by total_night_trips to find the high_traffic_zones:
- Final data out shows that the East Village and West Village had heavy traffic during night hours:

	LocationID	zone	pickup_count	dropoff_count	total_night_trips
0	79.0	East Village	23055.0	12234	35289.0
1	249.0	West Village	18451.0	7186	25637.0
2	48.0	Clinton East	15118.0	10151	25269.0
3	132.0	JFK Airport	20318.0	2943	23261.0
4	148.0	Lower East Side	14243.0	6488	20731.0
5	230.0	Times Sq/Theatre District	11882.0	6779	18661.0
6	68.0	East Chelsea	8965.0	8495	17460.0
7	114.0	Greenwich Village South	12919.0	3796	16715.0
8	107.0	Gramercy	8272.0	8353	16625.0
9	186.0	Penn Station/Madison Sq West	9987.0	5539	15526.0

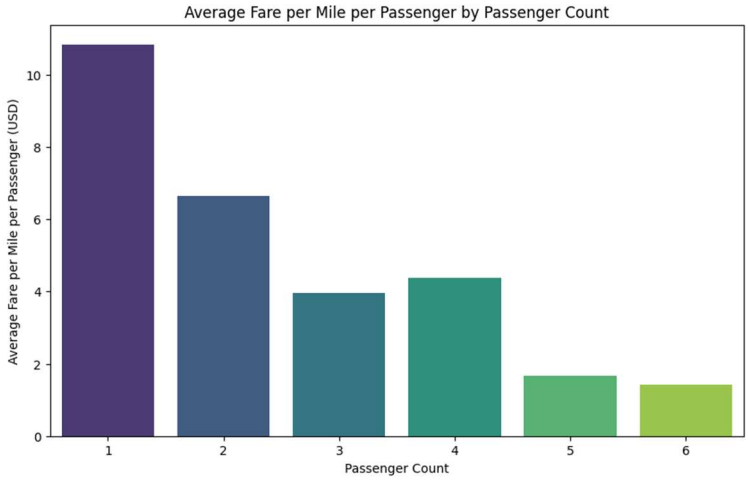
3.2.8. Find the revenue share for nighttime and daytime hours

- Daytime had higher revenue share when compared to night time. This was derived by defining a time_period as “Day” and “Night” based on the time and grouping the data based on the timeframe:
- Calculated the revenue by summing the fare_amount for each category and dividing by total_revenue.



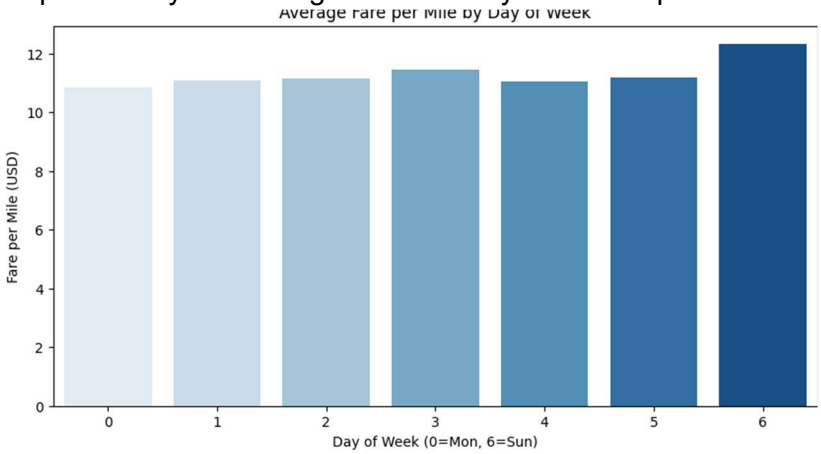
3.2.9. For the different passenger counts, find the average fare per mile per passenger

- Fare price gradually goes down when the number of passengers in the cab increases.



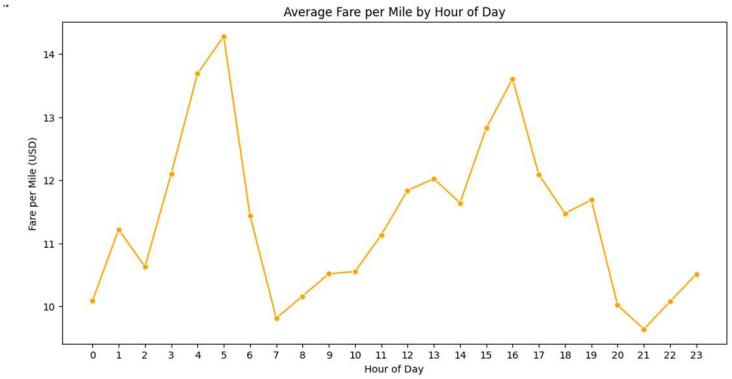
3.2.10. Find the average fare per mile by hours of the day and by days of the week

Fare per mile by hour is high on a Sunday when compared to other days.



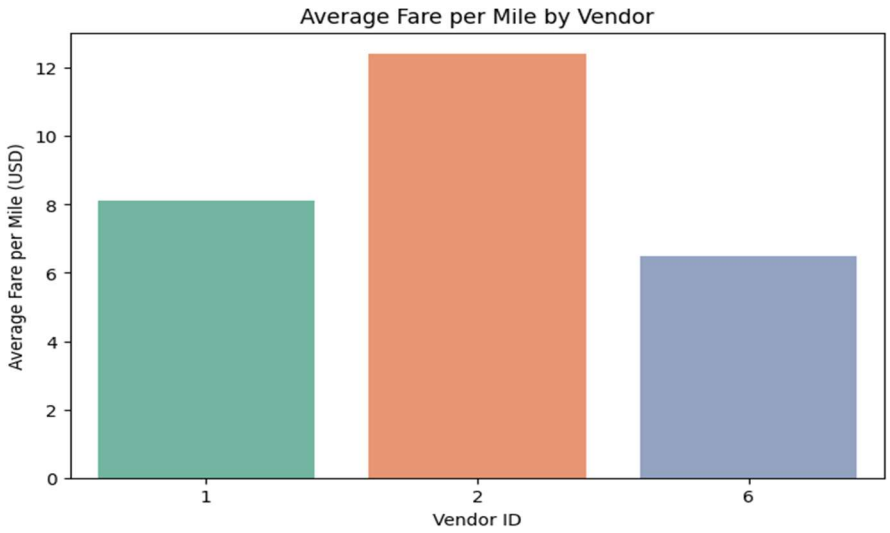
3.2.11. Analyse the average fare per mile for the different vendors

Average fare per mile is high early in the morning and drops at around 7 and gradually increases during the day:

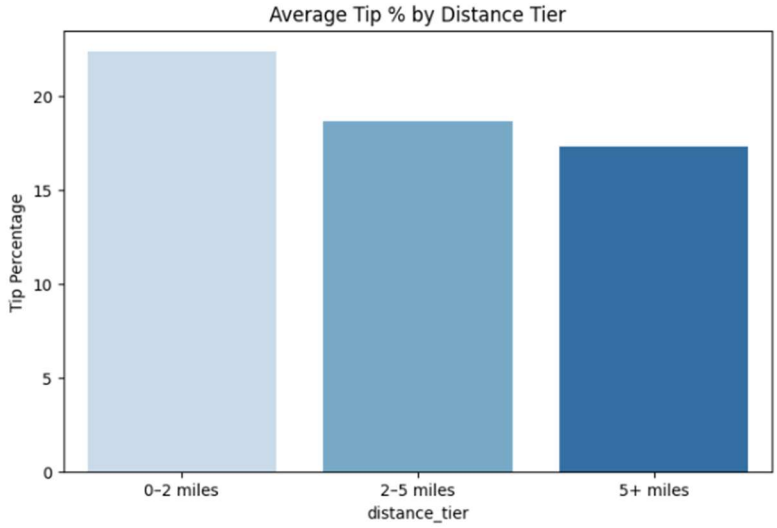


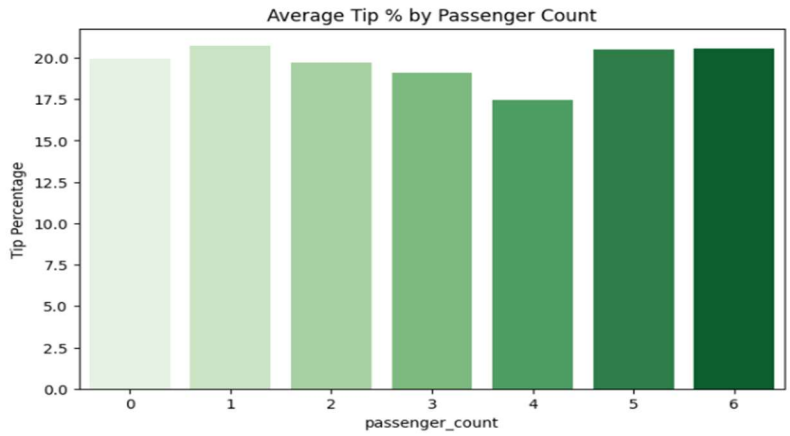
3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

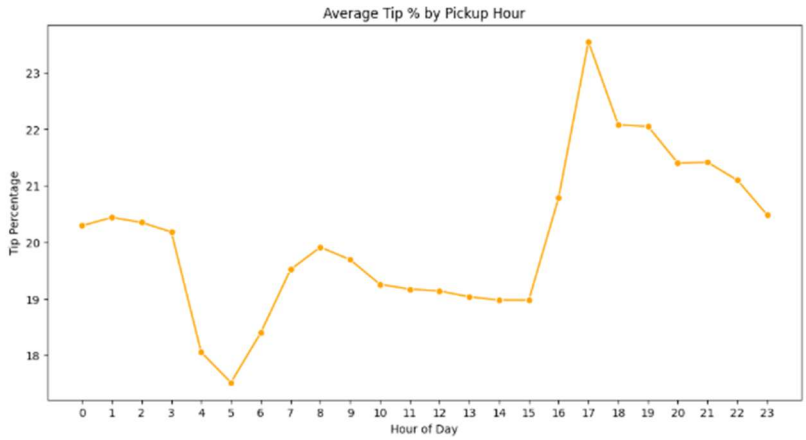
Fare rates are more for VendorID 2 (Curb Mobility, LLC) when compared to Vendor ID 1 (Creative Mobile Technologies, LLC) and VendorID 6 (Myle Technologies Inc) as shown in the below figure:



3.2.13. Analyse the tip percentages

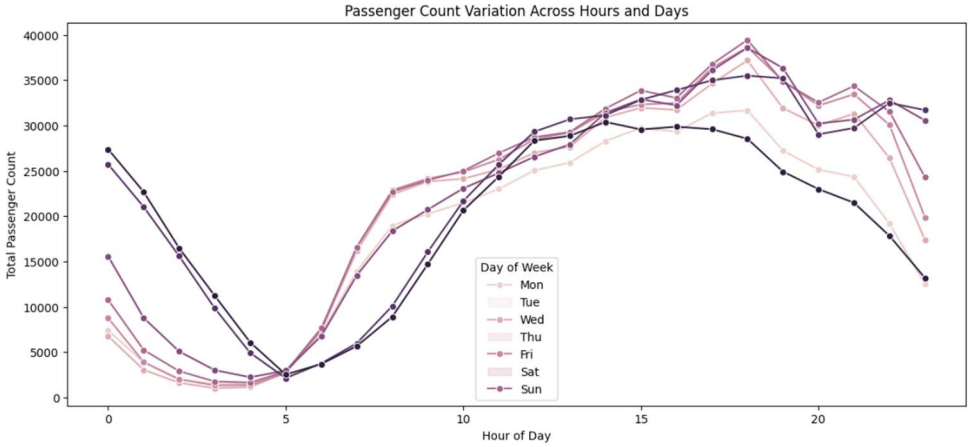






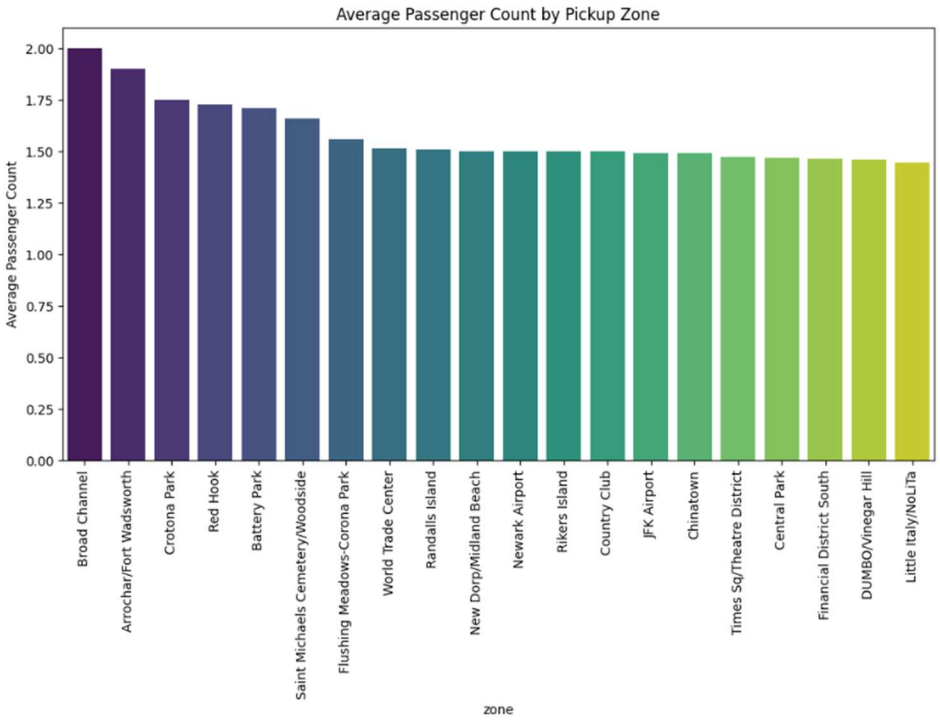
3.2.14. Analyse the trends in passenger count

Data clearly shows that weekend there are more people traveling in the midnight and the passenger count is less generally less in the morning on the weekend when compared to weekdays.



3.2.15. Analyse the variation of passenger counts across zones

This graph is drawn for the top 20 passenger count by zone



3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

- Congestion_surcharge is applied 93% of the time, which is calculated by counting the congestion_srucharge value not nan and determining the share:

	congestion_surcharge	count	share
0	2.50	2422495	93.112183
1	0.00	179198	6.887740
2	2.75	1	0.000038
3	0.50	1	0.000038

4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

To determine the recommendations below approach for used:

- Aggregated demand by zone, hour and day:

- Defined threshold for high demand and low demand zones based on average demand
- Did a strategic recommendation based on hour (morning commute/evening commute/late night commute)
- Created a function allocate_cabs to dynamically adjust supply based on demand thresholds:
- Merged demands with zones to get the zone names

Below table shows the top 15 allocation strategies for each hour with reference to day of week:

	zone	hour	dayofweek	trip_count_x	recommendation	allocation_strategy
0	Newark Airport	3	4	1	Maintain balanced distribution	Reduce supply by 20%
1	Newark Airport	3	5	1	Maintain balanced distribution	Reduce supply by 20%
2	Newark Airport	3	6	1	Maintain balanced distribution	Reduce supply by 20%
3	Newark Airport	4	0	1	Maintain balanced distribution	Reduce supply by 20%
4	Newark Airport	4	3	1	Maintain balanced distribution	Reduce supply by 20%
5	Newark Airport	5	4	1	Maintain balanced distribution	Reduce supply by 20%
6	Newark Airport	5	6	1	Maintain balanced distribution	Reduce supply by 20%
7	Newark Airport	6	1	3	Increase supply in residential zones	Reduce supply by 20%
8	Newark Airport	6	4	1	Increase supply in residential zones	Reduce supply by 20%
9	Newark Airport	6	5	1	Increase supply in residential zones	Reduce supply by 20%
10	Newark Airport	6	6	1	Increase supply in residential zones	Reduce supply by 20%
11	Newark Airport	7	5	1	Increase supply in residential zones	Reduce supply by 20%
12	Newark Airport	8	1	1	Increase supply in residential zones	Reduce supply by 20%
13	Newark Airport	9	1	1	Increase supply in residential zones	Reduce supply by 20%
14	Newark Airport	11	1	1	Maintain balanced distribution	Reduce supply by 20%

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

This was achieved following the below steps:

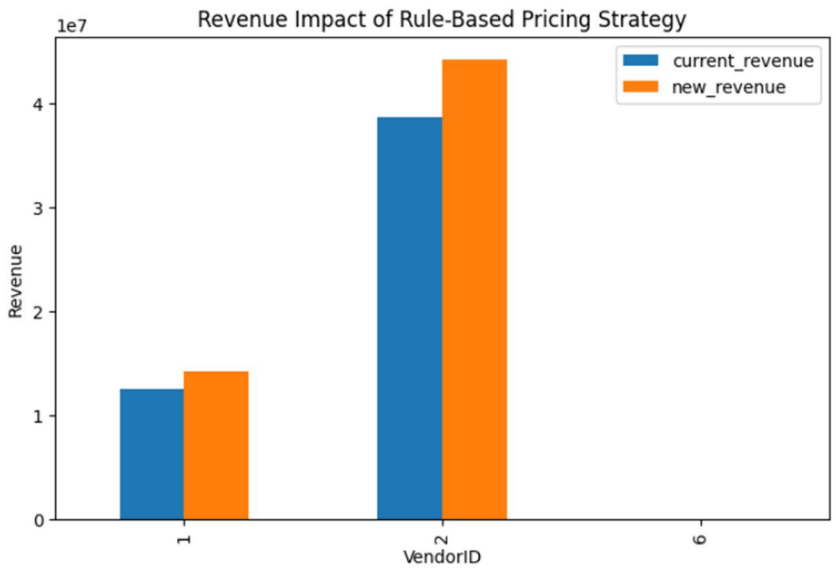
- Extracted the hour, dayofweek and month from the pickup_datetime:
- Aggregated the demand by location, hour,day and month
- Created a method positioning_strategy by setting positioning rules specific to morning commute, evening commute and late night commutes:
- Likewise created methods for day strategy (considering weekends), month strategy (considering holiday season during year end)
- Applied the strategy on the data and filtered data for time_strategy, day_strategy and month_strategy:
- Top 20 strategies are shown in the image below:

	zone	hour	dayofweek	month	trip_count_x	time_strategy	day_strategy	month_strategy
0	Newark Airport	3	4	1	1	Concentrate on nightlife zones and airports	Prioritize commuter flows (residential ↔ busin...	Maintain standard allocation
1	Newark Airport	3	5	4	1	Concentrate on nightlife zones and airports	Increase coverage in entertainment and tourist...	Maintain standard allocation
2	Newark Airport	3	6	5	1	Concentrate on nightlife zones and airports	Increase coverage in entertainment and tourist...	Maintain standard allocation
3	Newark Airport	4	0	10	1	Maintain balanced distribution	Prioritize commuter flows (residential ↔ busin...	Maintain standard allocation
4	Newark Airport	4	3	5	1	Maintain balanced distribution	Prioritize commuter flows (residential ↔ busin...	Maintain standard allocation
5	Newark Airport	5	4	9	1	Maintain balanced distribution	Prioritize commuter flows (residential ↔ busin...	Maintain standard allocation
6	Newark Airport	5	6	6	1	Maintain balanced distribution	Increase coverage in entertainment and tourist...	Increase coverage in residential areas (short ...
7	Newark Airport	6	1	5	1	Increase supply in residential zones and airports	Prioritize commuter flows (residential ↔ busin...	Maintain standard allocation
8	Newark Airport	6	1	8	1	Increase supply in residential zones and airports	Prioritize commuter flows (residential ↔ busin...	Maintain standard allocation
9	Newark Airport	6	1	11	1	Increase supply in residential zones and airports	Prioritize commuter flows (residential ↔ busin...	Boost airport and shopping district coverage
10	Newark Airport	6	4	11	1	Increase supply in residential zones and airports	Prioritize commuter flows (residential ↔ busin...	Boost airport and shopping district coverage

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

Below steps were taken to derive the data-driven adjustments to the pricing strategy to maximize revenue:

- Fare_per_mile is computed by dividing the fare_amount by trip_distance
- Extracted the hour and day_of_week from the pickup_datetime and defined a rule based pricing strategy defining the distance tier rule, time of day rule, day of week rule and location rules:
    - Setting the base_fare based on the trip_distance
    - Setting the base_fare based on the pick up time of the day
    - Setting the base_fare based on the day of the week (Weekend/Weekday)
- Applied the strategy on the dataset and calculated the optimal_fare_per_mile:
- Calculated current_revenue and new_revenue based on current fare_per_mile and optimal_fare_per_mile
- Created summary by grouping data based on vendor_id, current_revenue and new_revenue:


Revenue Impact of Rule-Based Pricing Strategy
1e7
current_revenue
new_revenue
Revenue
0
1
2
3
4
1
2
6
VendorID